

A Hybrid Deep Reinforcement Learning and LLM Framework for Real-Time Distillation Optimization

Integrating Rigorous DWSim Flowsheets with AI-Augmented Process Copilots

Suman Senapati

Master of Science in Applied Artificial
Intelligence

Shiley Marcos School of Engineering
/ University of San Diego
ssenapati@sandiego.edu

ABSTRACT

The abstract should be the last piece of your report that you write and will be a 1-2 paragraph overview of your entire project. As a rough guide, you should expect to have 1-2 sentences introducing the problem and your chosen dataset, 3-4 sentences describing your chosen methods, and 1-2 sentences summarizing your project results.

KEYWORDS

Deep Reinforcement Learning, Neural Networks, Deep Q-Networks, Soft Actor Critic, DWSim, Optimization, AI Agent, Process Simulation

1 Introduction

1.1 Problem Statement

Crude Distillation Units (CDUs) are the foundational and most energy-intensive separation processes in petroleum refining. Crude Oil is processed in an oil refinery through multiple Unit Operations, to produce various finished products like LPG, Motor Spirit (Gasoline), Jet Fuel, Diesel, Low-Sulphur Fuel Oil, Bitumen and so on. The crude is first fractionated into multiple unfinished

intermediates which are further refined into various finished products in subsequent processes. Traditionally, Advanced Process Control (APC) strategies, such as Model Predictive Control (MPC), have been utilized to maintain process variables within strict operational bounds. However, APC models are linear and bound by standalone strategies and lack ability to run holistic optimization. As refining margins become increasingly volatile, there is a critical need to transition from steady-state regulation to dynamic, market-driven overall optimization. While Deep Reinforcement Learning (DRL) has emerged as a powerful paradigm for solving highly non-linear, stochastic optimization problems in chemical processes (Spielberg et al., 2019), its industrial implementation remains severely hindered by the ‘black-box’ nature of neural networks. Operators and process engineers are inherently reluctant to deploy opaque control policies that lack physical interpretability (Venkatasubramanian, 2019). This study seeks to develop a solution which utilizes Deep Reinforcement Learning coupled with process Digital Twin, for economic and process optimization of Crude Distillation Unit.

1.2 Significance

This study bridges the critical gap between advanced artificial intelligence and practical industrial operability. By maximizing thermodynamic efficiency and product yield in response to fluctuating market prices, refineries can realize substantial economic gains while minimizing energy waste and emissions. Additionally, introducing an LLM-based cognitive AI agent as an interpretability and reporting layer addresses the operator trust deficit. This ‘Explainable AI’ (XAI) approach ensures that human operators remain in the loop, capable of auditing and understanding the physicochemical rationale behind the AI’s suggested setpoint changes (Nian et al., 2020).

1.3 Target End-Users

The primary end users of this hybrid AI framework are chemical plant operators, process engineers, and production planners. While the Deep RL agent performs high-dimensional mathematical optimization in the background, the LLM interface is designed specifically to communicate with these domain experts in natural language, translating thermodynamic state changes into actionable engineering reports.

1.4 Data Sources and Simulation Context

Reinforcement learning agents learn from experience while navigating through the internal environment. The training data for neural network to compute Q-value is generated from the simulation.

Various crude assay data is used for process simulation to ensure a robust model able to handle various types of feed quality (ExxonMobil, n.d.)

1.5 Final Deliverable

This study aims to successfully design, train, and deploy an AI-augmented digital twin for a CDU. The application has a Django dashboard to interact with the backend system comprising of a process Digital Twin built on DWSim, an optimizer built on Deep Reinforcement Learning agent, and a LLM-based AI Agent which creates reports and interacts with the user for explainability. In the interface, the user will input current market prices, triggering the Deep RL agent to interact with the underlying DWSim physical model to discover optimal operating parameters. The final output is a set of optimized targets for the plant to operate and a comprehensive, natural-language engineering report generated by the LLM, detailing the economic trade-offs, thermodynamic constraints, and justification for the proposed optimization.

2 Data Summary

2.1 Input State Representation

Our study revolves around finding the most optimized manner of distillation of crude. Hence, one of the primary focuses in this study is to understand the properties of commonly available crude assays. The dataset chosen comprises physicochemical properties for five distinctive crude oil assays (Azeri Light, Erha, Tapis, Upper Zakum, and WTI Light) sourced from the ExxonMobil’s open database (ExxonMobil, n.d.).

Whole Crude Physicochemical Attributes such as mass density, API gravity (the standard measure of extent of heaviness of oil), Total Sulfur content, and Total Acid Number (TAN) are critical defining factors of crude assays. These properties are critical as they define the intrinsic quality and potential corrosivity of the feed (González-Durán et al.,

2024; Al-Sayegh et al., 2016). Concentrations of Nitrogen and metals such as Vanadium (V) and Nickel (Ni) are significant since they serve as catalyst poisons in downstream units like fluid catalytic cracking (FCC) (Al-Sayegh et al., 2016). Rheological and Volatility Markers such as Kinematic Viscosity, Reid Vapor Pressure (RVP), and Pour Point characterize the flow behavior and thermal stability of the crude. The True Boiling Point (TBP) curve defines the volumetric and weight percentage yields for various product cuts (Naphtha, Kerosene, Diesel, and Residue). Quality metrics for these cuts—such as Octane Number (RON/MON) for Naphtha and Cetane Index for Diesel — are utilized as process constraints within downstream process optimizers (AlRijeb et al., 2025).

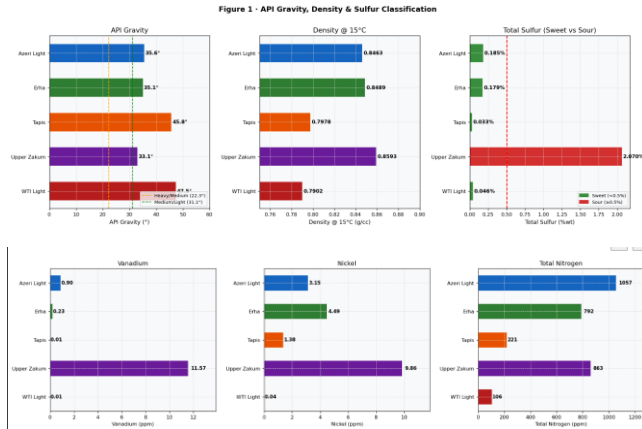


Fig 1. (a). API Gravity, Density (at 15 °C) and Total Sulphur distribution in the selected crude assays

Fig 1. (b) Metals and Total Nitrogen content in the selected crude assays

2.2 Fractional Yield Dynamics and TBP Analysis

Upon analysis, True Boiling Point (TBP) distillation curves derived from the dataset shed light on significant disparities in product slates. WTI Light and Tapis have demonstrated superior

Naphtha yields, which are essential for gasoline production. Conversely, Erha and Tapis have showed a higher propensity for middle distillate production like Kerosene and Diesel. Upper Zakum is characterized by a bottom-heavy profile, yielding a higher percentage of Vacuum Gas Oil (VGO) and atmospheric residues. Since the Deep RL agent is inherently aimed at driving margins, yield distributions directly impact the reward function of the optimizer; for instance, when market prices favor Jet Fuel, the optimizer must identify the specific operating temperatures within the digital twin that maximize the Kerosene cuts while maintaining thermodynamic stability (Li et al., 2025).

	Light Ends (C5-60°C)	Light Naphtha (60-100°C)	Med. Naphtha (100-140°C)	Kerosene (140-200°C)	Jet Fuel (200-240°C)	Diesel (240-280°C)	10-600 (280-380°C)	Heavy Aro. (380-510°C)	Atm. Residue (510°C+)	VGO (370-480°C)	HVGO (480-580°C)	HVGO (580-680°C)	Vis. Residue (680°C+)	TOTAL
Azeri Light	2.86	4.35	6.14	8.64	9.88	11.01	10.71	9.99	10.46	14.21	7.18	5.59	12.40	103.33
Erha	2.10	2.30	4.36	5.24	11.62	10.22	12.27	4.33	10.70	14.24	5.46	2.94	6.31	102.29
Tapis	3.96	7.16	12.42	19.89	15.17	11.86	10.24	3.24	10.26	11.01	4.10	2.52	4.79	111.09
Upper Zakum	4.30	1.74	7.37	6.31	6.27	6.34	6.20	3.30	46.14	12.81	7.60	6.72	16.11	140.13
WTI Light	5.96	10.22	16.34	12.24	10.21	9.46	6.20	2.89	21.87	5.57	4.43	3.16	4.64	110.63

Fig 2. Crude-wise yield distribution (wt%)

2.3 Contaminant Profiles and Catalyst Poisoning Potential

The comparative study between the selected crude assays indicates towards a strong correlation between the density of the crude and the concentration of catalyst poisons, specifically Vanadium, Nickel, and Micro-Carbon Residue (MCR). WTI Light and Tapis are identified as the cleanest feedstocks, with minimal metal content, making their heavy fractions ideal for Fluid Catalytic Cracking (FCC) units. Whereas, high levels of metals and asphaltenes in Upper Zakum and Azeri Light may pose a significant challenge for downstream catalyst deactivation (Al-Sayegh et al., 2016).

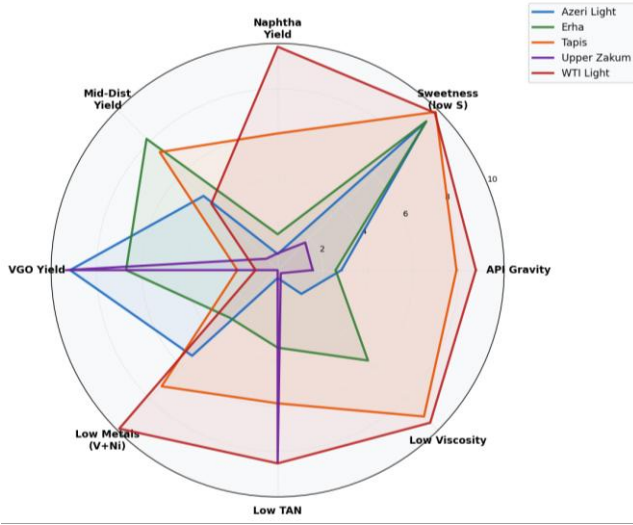


Fig 3. Radar-chart showing comparative study of major factors of crude quality

2.4 Economic Feedstock Suitability and Decision Optimization

Economic trade-offs associated with each assay's quality studied in detail indicates that Tapis and Erha provides an optimal balance with Erha seemingly more inclined towards for diesel-centric production. Upper Zakum, despite its high impurity levels, may offer higher margins if refinery can effectively manage its heavy-end processing. While WTI Light offers the highest yield of high-value light ends and the lowest treating costs, it typically commands a market premium. The data suggests that integration of a Large Language Model (LLM) as an interpretability layer is essential here; it translates these complex thermodynamic and economic trade-offs into natural language engineering reports. This Explainable AI (XAI) layer is likely to shed light to understand why the agent might suggest a more difficult-to-process feed like Upper Zakum during periods of specific market volatility (Wellawatte & Schwaller, 2025).

Crude	Classification	Origin	API	Dollar (t/bbl)	CDU/FIN Processability	Refinery Feed Quality	FTC Feed Quality	Hydrocracker Demand	Jet Fuel Potential	Steam Cracker Quality	Petrochemical Feed	Other/Feed Challenge
Azeri Light	Light Sweet	Azerbaijan	35.6	0.185	5	4	5	2	4	4	4	5
Erha	Light Sweet	Algeria	35.1	0.179	5	4	3	3	4	5	4	2
Tapis	Light Ultra Sweet	Malaysia	45.8	0.033	5	4	4	5	5	5	4	4
Upper Zakum	Light Sour	UAE	33.1	2.070	4	2	2	1	3	2	3	1
WTI Light	Light Ultra Sweet	Texas, USA	41.5	0.046	5	4	5	5	5	5	4	5

Fig 4. Crude-wise comparison based on its likely process and economic impact on a typical refinery as a whole

3 Literature Review

The traditional hierarchy of refinery control inherently resembles a temporal decoupling between the Real-Time Optimization (RTO) and Advanced Process Control (APC) layers, where the RTO typically samples unit states and recalculates economic targets, subject to model convergence, at a set interval of one to two hours. This multi-hour sampling gap, combined with the significant computational latency inherent in mathematical optimizers reaching convergence for complex non-linear models, forces the APC layer to manage transient process behavior and disturbances without updated economic guidance, often resulting in sustained periods of suboptimal operation. Recent advancements in Dynamic Real-Time Optimization (DRTO) attempt to bridge this gap. However, the computational cost of solving a rigorous, first-principles model in real-time still remains. This is where the integration of simulation model with a Deep RL agent offers a superior alternative. During training phase, by treating the simulation model as a high-fidelity simulator, the agent learns a control policy $\pi_{\theta}(a|s)$ that directly maps plant observations to optimal setpoints. Unlike the Sequential Quadratic Programming (SQP) methods described by Biegler (2010), which require a derivative-based search through the variable space at every time step, the Deep RL agent performs an instantaneous inference, significantly reducing the latency.

The predominant paradigm in industrial Real-Time Optimization (RTO) has historically relied on Non-Linear Programming (NLP) and Successive Quadratic Programming (SQP) to solve steady-

state models. While mathematically rigorous, these ‘equation-oriented’ solvers are often criticized for their ‘rigidity and susceptibility to convergence failures’ when faced with the high dimensionality of Crude Distillation Units (CDUs) (Biegler, 2010). A recurring deficiency in these mathematical optimizers is their inability to capture transient dynamics without prohibitive computational overhead (Darby et al., 2011).

A persistent challenge in mathematical RTO is the trade-off between profitability and the ‘Safe Operating Envelope’. Mathematical optimizers often operate at the ‘active constraints’ (Darby et al., 2011), leaving little room for error during stochastic disturbances. The shift toward Safe Reinforcement Learning, as highlighted by the emergence of the Constrained Policy Optimization (CPO) and Lagrangian RL frameworks (Achiam J., 2017), represents a significant trend in recent academic literature.

The integration of DWSIM—a CAPE-OPEN compliant, first-principles simulator—serves as a critical corrective. By utilizing DWSIM as the ‘ground truth’ environment, the DRL agent is not merely performing statistical curve fitting; it is interacting with a thermodynamically consistent system governed by Equations of State. This ‘First-Principle Compliance’ ensures that the learned control policy $\pi(a|s)$ remains bounded by physical reality, effectively bridging the gap between pure data-driven machine learning and rigorous chemical engineering (Spielberg et al., 2019).

In the context of a CDU, the state space S is characterized by a high-dimensional vector containing stage temperatures T_i , pressures P_i , mass fractions of pseudocomponents across the distillation curve, and product yields.

A critical contribution by Wang et al. (2024) in the *Journal of Process Systems Engineering* highlights the importance of Feature Engineering in RL for refineries. It is argued that normalizing states relative to the Feed Assay (the specific TBP curve of the incoming crude) allows the agent to generalize its learning across different crude blends—a concept very similar to well-known Transfer Learning where a model’s weight from learned from a larger dataset is partially fine-tuned for a specific domain use-case.

The reward function R_t is the most critical design element for any optimizer, which may be formulated as:

$$R_t = \sum_{p \in \text{Products}} (m_p \cdot V_p) - \sum_{u \in \text{Utilities}} (m_u \cdot C_u) - \sum_{c \in \text{Constraints}} \lambda_c \cdot \max(0, g_c(s) - \text{limit}_c)$$

Here, V_p represents the market value of product p , C_u is the cost of utilities (steam and cooling water), and λ_c is a penalty coefficient for violating quality constraints like Flash Point or ASTM D86 95% points. Literature review of recent times shows use of Lagrangian multipliers within the RL framework to dynamically adjust these penalties, ensuring that the agent prioritizes safety and product specifications over raw yield during periods of high process volatility (Stooke A., et al., 2020).

Academic research on Constrained Policy Optimization (CPO) algorithm shows that it is used to ensure that the agent remains within the Safe Operating Envelope of a VDU-ADU sequence. Their findings suggest that while CPO may converge slightly slower than standard PPO, it effectively eliminates the risk of dry trays or

column flooding during the learning process (Achiam L., et al., 2017).

While the RL agent maximizes the profit-based reward function R_t , it often lacks the semantic awareness to identify nuanced process risks, such as dew point corrosion. Traditional RTOs focus on economic optima and may push the column toward temperatures that inadvertently lead to the condensation of acidic water in the overhead system. Recent studies in 2025 have proposed the use of Large Language Models (LLMs) as ‘Cognitive AI Agents’ that sit atop the optimization layer (Bran A. M., et al., 2024).

4 Methodology

4.1. Process Simulation

The process simulation is developed using DWSIM, a CAPE-OPEN compliant flowsheet simulator, utilizing a comprehensive model that encompasses an Atmospheric Distillation Unit (ADU), a Naphtha Stabilizer Unit (NSU), and a Vacuum Distillation Unit (VDU). This multi-unit sequence is configured to generate 13 distinct product streams, ranging from light gases to heavy residues. The ADU section produces five streams including un-condensed gas, Unstabilized Naphtha, Heavy Naphtha (HN), Straight-Run Kerosene Oil (SKO), Light Gas oil (LGO), and Heavy Gas oil (HGO). The NSU further refines light components of Unstabilized Naphtha into stabilizer off-gas, LPG, and stabilized run naphtha (SRN), while the VDU yields vacuum off-gas, vacuum diesel, Vacuum gas oil (VGO), Hotwell oil, and vacuum residue, while feeding from bottoms of ADU which is referred to as Reduced Crude Oil (RCO). For academic purpose, the flowsheet was converged on Erha Crude assay. The flowsheets developed are shown in Fig 4.

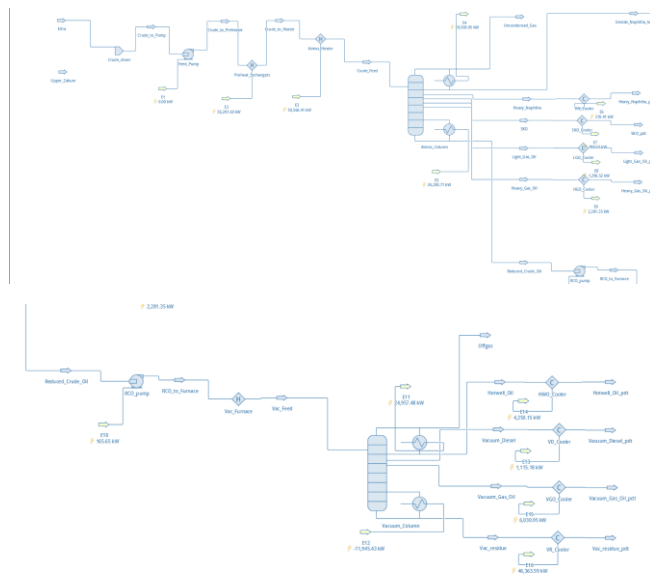
The crude feed is defined at ambient temperature and atmospheric pressure which is then pressurized through booster pumps, and heated in two phases

upto 360 °C where 56% of the crude turns in vapour phase.

The Atmospheric Distillation Unit (ADU) is configured with 30 stages, at 1.7 bar Top pressure with 0.4 bar pressure drop. Four side drawn products are defined. A partial condenser with initial reflux ratio of 5 used. Bottom temperature is set at 365 °C. The overhead liquid stream, Unstabilized Naphtha is drawn and fed to Naphtha Stabilizer to separate LPG from Straight-run Naphtha. The bottom, Reduced Crude Oil (RCO), is fed to Vacuum Distillation Unit for further refinement, after heating upto 405 °C which vaporizes 52% of the feed.

Naphtha Stabilizer (NSU) is configured with 30 stages, at 9.5 bar with 0.4 bar pressure drop, a partial condenser with 0.05 bar pressure drop, reflux ratio of 5 and bottom temperature at 155 °C.

Vacuum Distillation Unit (VDU) is configured with 20 stages. The top pressure is set at 0.2 bar to resemble a near-vacuum environment, while the column pressure drop is configured at 0.5 bar. A partial condenser is configured with a pressure drop of 0.1 bar and a reflux ratio of 5. The bottom temperature is set at 360 °C.



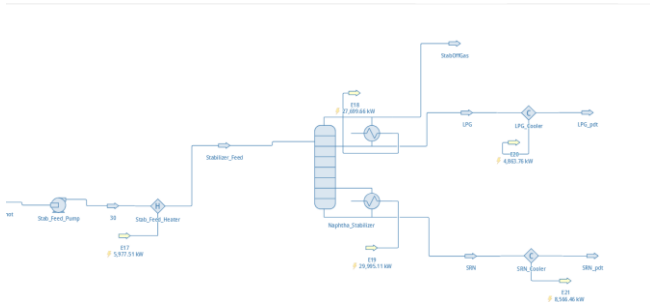


Fig 4. Flowsheets created depicting a typical Crude Distillation Unit

4.2. Reinforcement Learning Architectures

4.3. DWSim-RL Bridge

4.4. Agent Training

4.5. Model Optimization

4.6. AI Agent Creation

4.7. Application development

This section of your final report should describe the final model architecture or pipeline choices that you have made, as well as the training and optimization procedures that you follow. Training results should be saved for the Results section of the paper—any discussion of modeling iterations and optimization should be brief, without any mention of the resulting performance. For example: “During model optimization, we explored hyperparameters a , b , and c with values $[d,e,f]$, $[g,h,i]$, and $[j,k,l]$, respectively. Our final model architecture was...” If the comparative results over your hyperparameter search are interesting enough to be included, these should be discussed in the results section.

The reader should be able to answer the following questions after reading your methodology:

- What type of machine learning model(s) are you using?
- What are the notable architectural or system design choices that you have made and why? E.g., number of nodes, hidden layers, activation functions, etc.
- What is your model training procedure or machine learning pipeline? E.g., How did you split your data? How many epochs/iterations did you use during training? What was your batch size? What loss function/training metrics were used?
- How did you optimize your model? E.g., Which hyperparameters and/or architectural elements did you adjust?

5 Results

This section of your final report should describe how your model/system performed and provide evidence for how your work did or did not support your experimental hypothesis or solve your chosen problem. The reader should be able to answer the following questions:

- How effective was your machine learning model at learning the task? e.g. Did the model overfit/underfit the training data? What does the training accuracy/loss curve look like for your model?
- If your project is focused on answering a research question, what evidence do you have to support or disprove your hypothesis?
- If your project is focused on solving a problem, how will your model performance affect the application of your model to that problem?
- If your project is focused on implementing a complete machine learning system, what is the user experience? Are there any delays

or remaining bugs that affect your model output either in common or edge use cases?

6 Conclusion

This section of your final report should provide a very brief summary of your entire project. It should highlight the most interesting or significant results and discuss any future work that you or another researcher might do. The reader should be able to answer the following questions:

- What was the main hypothesis or goal of the project, and how did your models perform in relation to this hypothesis/goal?
- What was the most interesting or significant result of your project?
- Did your model training or performance bring about any unexpected results?
- If you were to continue this project, what would your next steps be? For example, would you continue to optimize your model or try other model architectures? Would you change or expand the data used in your model? What steps need to be taken to “productionize” your model?

7 FORMATTING EXAMPLES

DELETE THIS SECTION BEFORE FINAL SUBMISSION

7.1 General Formatting

A formal report of this type should be primarily in paragraph form, with minimal subheadings, bullet points, or numbered lists. For most of your required report sections, no subheadings will be necessary, and the information can be presented as a single, cohesive discussion of the content relevant to that section. As a general rule of thumb, bulleted or numbered lists are only appropriate in this kind of report if the list items are less than one sentence, but are too long/detailed to be presented in a table.

If you feel strongly that your content would benefit from subheadings or bulleted/numbered lists, they should follow the formatting examples used in this section.

7.2 In-text Citations

A report of this type should be heavily supported by in-text citations. You should include a new supporting citation in each section for:

1. The first time that you mention a specific code library or dataset,
2. The first time that you introduce a pre-trained model or model architecture,
3. Any information that is not your own original thought/idea, even if you didn't need to look it up.

When in doubt, it is always better to over-cite than to under-cite.

Formatting rules for in-text citations can be found in the APA guidelines, but examples of the three most common uses are discussed in the following paragraphs.

Parenthetical citations at the end of a sentence should be used when the information from the entire sentence comes from the cited source (American Psychological Association, 2025).

Parenthetical citations mid-sentence should be used when only a part of the sentence comes from the cited source, or when you are providing a citation for something specific such as a company, model architecture/method, or technique. See the following examples for a sentence with multiple mid-sentence citations and a sentence with a specific mid-sentence citation below:

- The transformer model was introduced in 2017 for the application of machine translation (Vaswani et al., 2017) and quickly became the most common backbone for today's large language

models (Devlin et al., 2019; Radford et al., 2019).

- As one of the first transformer-based generative text models, OpenAI’s GPT (Radford et al., 2019) quickly became the standard large language architecture used for generative tasks.

Narrative in-text citations can be used when you want to include the name of the author(s) in your main text, as shown in the following example:

Vaswani et al. (2017) originally developed the transformer architecture for the application of machine translation.

7.3 **Figures, Tables, and Equations**

Most of your supporting content should be included in the main text of the report, rather than at the end. While the in-line formatting can be a headache, this makes it easier for your reader to reference your figures and tables while reading, without having to scroll back and forth too much. See Section 7.5 for more details about how to properly use appendices for supplemental information.

7.3.1 *Figures*

Figures should be numbered sequentially *below* the image and referred to directly in your text (e.g. “As seen in Figure 7.1 [...]”, “Figure 7.1 shows [...]”). If your figure is important enough to include in your report, then it deserves at least one sentence of discussion in the text.

Each figure should also have a corresponding caption that briefly describes the content of the figure and can be used to highlight important details. If your figure is too complex to be limited to a single column width, you can include it as a full-width image at the top of the page.



Figure 7.1

Example text: Model accuracy for our six models during training. The best performing model is shown in red.

7.3.2 *Tables*

Tables should be numbered sequentially above the table and should have an in-text mention and a descriptive caption, like the figure examples given above.

Feel free to improve the default MS Word table formatting shown here for aesthetic appeal (e.g. change cell size, border width, font size, etc.). If you have a favorite table formatting tool (e.g. LaTeX, Pandas pretty print, etc.), feel free to use that and include the output as an image. However, please ensure that the image quality is legible and the formatting generally matches the rest of the report, and DO NOT include screenshots of raw code output—it is not professional to include dark mode or Courier New code output, or screenshots of your console window.

Table 7.1

Example text: Results for all three models over the standard benchmarking metrics. Highlighted cells show the best performance for each metric.

A table is often a better alternative to a bulleted list for communicating information that may otherwise seem redundant (e.g. variable details or model hyperparameter choices), or to make information easier to compare across categories (e.g. model performance). If you have a very complex table, think about how you might use bold or italic font to highlight notable cells, such as the best performing model.

7.3.3 Equations

Think carefully about whether including an equation is appropriate for your semi-technical stakeholder audience—most of the time that level of detail will be too technical, and equations should either be left out or included in a separate appendix.

If you decide that you do want to include an equation, they should be on a separate line, indented, and numbered sequentially to the right of the equation. As with figures and tables, equations should be referenced directly in your text with a short description (e.g. “Equation 7.1 shows the basic linear regression equation, where y is the target variable, x is the model input, a is the learned coefficient, and b is the model intercept.”) You DO NOT need to include a separate caption for your equations.

$$y = ax + b \quad (7.1)$$

As with tables, if you have a favorite tool for formatting your equations, feel free to include the formatted equation as an image, as long as it matches the formatting shown in the example above.

7.4 Works Cited

References should be alphabetically ordered using APA 7 formatting. You can refer to the [APA guidelines](#) on how to format citations from different types of sources. There are also online tools that can be used to create citations for you,

though these will occasionally produce errors, so be sure to double check these. If you are citing an academic paper, the Google Scholar listing for the paper will also have a properly formatted APA citation that you can copy into your report. Several example citations are shown in the works cited included in this template.

7.5 Appendices

Appendices can be used to include information that you feel is important to the communication of your project but may be too technical or too detailed for the intended semi-technical stakeholder audience. This may include items like a detailed description of data transformations, feature engineering, or architectural choices, or complete results over a hyperparameter grid search or a series of experimental architectures.

Appendices are not included in the page count but should not be used to get around the report page limits. Information included in the appendices should not be necessary for a basic understanding of your project and should instead provide deeper detail for an interested reader. You can assume that your stakeholder audience would not read most of the content in an appendix, and as such, your appendix content may not contribute to your final report grade.

Appendices should be labeled alphabetically (Appendix A, Appendix B, etc.) and should be referred to directly in your text (e.g. “A complete table of results over all hyperparameters can be found in Appendix A”). Each appendix should start on a new page and does not need to be in the two-column format if full page width makes more sense for your content (e.g. detailed figures or large tables).

ACKNOWLEDGMENTS

These are optional, but might mention any external contributors to your project, such as professors or mentors who provided data or consultation.

Works Cited

- Achiam, J., Held, D., Tamar, A., & Abbeel, P. (2017). Constrained Policy Optimization. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 70, 22–31.
- AlRijeb, M. F. S., Othman, M. L., Ishak, A., Hassan, M. K., & Albaker, B. M. (2025). Machine learning-driven soft sensor implementation for real-time fault detection in CDU of oil refinery. *Engineering, Technology & Applied Science Research*, 15(1), 20425–20432. <https://doi.org/10.48084/etasr.9781>
- Al-Sayegh, A., Al-Wahaibi, Y., Joshi, S., Al-Bahry, S., Elshafie, A., & Al-Bemani, A. (2016). Bioremediation of heavy crude oil contamination. *The Open Biotechnology Journal*, 10(1), 301–311. <https://doi.org/10.2174/1874070701610010301>
- Biegler, L. T. (2010). Nonlinear programming: Concepts, algorithms, and applications to chemical processes. Society for Industrial and Applied Mathematics (SIAM). <https://doi.org/10.1137/1.9780898719383>
- Bran, A. M., Cox, S., Edwards, O., Diggans, C., Schwaller, P., & White, A. D. (2024). ChemCrow: Augmenting large-language models with chemistry tools. *Nature Machine Intelligence*, 6(6), 525–535. <https://doi.org/10.1038/s42256-024-00832-w>
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. arXiv. <https://doi.org/10.48550/arXiv.1606.01540>
- Darby, M. L., Nikolaou, M., Jones, J., & Nicholson, D. (2011). RTO—An overview and assessment of current practice. *Journal of Process Control*, 21(6), 874–884.
- Engell, S. (2007). Feedback control for optimal process operation. *Journal of Process Control*, 17(3), 203–219. <https://doi.org/10.1016/j.jprocont.2006.10.003>
- ExxonMobil. (n.d.). *Crude oil assays*. <https://corporate.exxonmobil.com/what-we-do/energy-supply/crude-trading/crude-oil-assays>
- González-Durán, R., Rodríguez-Prieto, A., Camacho, A. M., Peña-Ballesteros, D. Y., & Serna, A. (2024). Experimental assessment of corrosion properties for materials intended for heavy crude processing. *Materials*, 17(21), 5275. <https://doi.org/10.3390/ma17215275>
- Hubbs, C. P., Li, C., Sahinidis, N. V., Grossmann, I. E., & John, J. M. (2020). A deep reinforcement learning approach for chemical production scheduling. *Computers & Chemical Engineering*, 141, 106982. <https://doi.org/10.1016/j.compchemeng.2020.106982>
- Li, Z., Lin, R., Su, H., & Xie, L. (2025). Reinforcement learning-driven plant-wide refinery planning using model decomposition. *arXiv*. <https://doi.org/10.48550/arxiv.2504.08642>
- Nian, R., Liu, J., & Huang, B. (2020). A review on reinforcement learning: Introduction and applications in industrial process control. *Computers & Chemical Engineering*, 139, 106886.
- Qin, S. J., & Badgwell, T. A. (2003). A survey of industrial model predictive control technology.

- Control Engineering Practice, 11(7), 733–764.
[https://doi.org/10.1016/S0967-0661\(02\)00186-7](https://doi.org/10.1016/S0967-0661(02)00186-7).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv.
<https://doi.org/10.48550/arXiv.1707.06347>
- Shin, J., Badgwell, T. A., Liu, K. H., & Lee, J. H. (2019). Reinforcement learning – Overview of recent progress and implications for process control. *Computers & Chemical Engineering*, 127, 282–294.
<https://doi.org/10.1016/j.compchemeng.2019.05.029>
- Spielberg, S., Cao, Y., & Gopaluni, B. (2019). Deep reinforcement learning for process control: A review for control engineers. *Control Engineering Practice*, 89, 212–222.
<https://doi.org/10.1016/j.conengprac.2019.05.017>.
- Spielberg, S., Tulsyan, A., Lawrence, N. P., Loewen, P. D., & Bhushan Gopaluni, R. (2019). Toward deep reinforcement learning for process control: A review. *Computers & Chemical Engineering*, 132, 106602.
- Spielberg, S., Tulsyan, A., Lawrence, N. P., Loewen, P. D., & Bhushan, G. (2019). Toward self-driving processes: A deep reinforcement learning approach to control. *AIChE Journal*, 65(10), e16733.
<https://doi.org/10.1002/aic.16733>.
- Stooke, A., Achiam, J., & Abbeel, P. (2020). Responsive safety in reinforcement learning by PID Lagrangian methods. *Proceedings of the 37th International Conference on Machine Learning (ICML)*.
- Venkatasubramanian, V. (2019). The promise of artificial intelligence in chemical engineering: Is it here, finally?. *AIChE Journal*, 65(2), 466–478.
- Wellawatte, G. P., & Schwaller, P. (2025). Human interpretable structure-property relationships in chemistry using explainable machine learning and large language models. *Communications Chemistry*, 8, Article 11733140.
<https://doi.org/10.1038/s42004-024-01393-y>